

The privacy layer — design

PQ wallets, ring-authorized authority, and network privacy inside PQGuard.

Status: design. Written 2026-09-07. Extends [pqguard-spec.md](#) and [nonce0-complete-design.md](#); changes neither's existing claims.

1. The threat class this adds

The current threat model covers key **recovery**. It does not cover target **selection**, and those are different problems with different fixes.

A quantum adversary has to choose. Shor on secp256k1 is not free even after a CRQC exists — it is scheduled time on scarce hardware, so keys get broken in return-on-investment order.

Every protocol today publishes a perfectly sorted target list, for free: `owner()`, the EIP-1967 admin slot, `SafegetOwners()`, and balances to rank by. That list is exactly what nonce0 computes, which is why a full kill-chain report **is an attack plan** and why the scan API is tiered rather than open.

PQGuard makes a broken key **insufficient**. This layer makes the holder **unfindable**. The two multiply: the adversary must both pick the right target and defeat a second factor on it.

	Threat	Mechanism	Cost
A	The target list is public and value-sorted	ring-authorized authority	expensive
B	The escape transaction is a public race	network privacy at broadcast	~free
C	The signer set is enumerable — 3 keys, 2 suffice	PQ wallets + rings	cheap / expensive

Do B first. It is nearly free, it closes the window that actually kills protocols during a migration, and it needs no new cryptography. A and C are second-order optimisations that cost real gas. Ranking them the other way round is the expensive mistake.

2. Threat B — the mempool is a kill zone

The dangerous period is not Q-day. It is the months between "quantum is real" and "everyone has migrated," during which **every defensive transaction you broadcast is an attack opportunity**. You send a rotation; it sits in the mempool; an adversary holding your already-broken key sees it and races you with a higher fee.

Two defences, and the design uses both:

Arm before the break. Already the design in [nonce0-complete-design.md](#) §7. The ECDSA-authorized step happens while ECDSA is still trustworthy, so at Q-day there is no race left to lose — the destination is already committed and the authorizing signature is hash-based.

Relay mesh at broadcast. Batching plus randomized delay across a fixed 3-hop path means an adversary watching the mempool cannot tell which transaction is yours, or when one is coming. A public race becomes a sealed bid.

Which transactions go through the mesh:

Transaction	Through the mesh	Why
registerKey (init)	yes	reveals that this account is preparing a defence
transferOwnership to the adapter	yes	the 3-transaction install window is the exposed one
arm(digest, sig)	yes	reveals the intent and the timing
execute(...)	yes	the race being defended against
Ordinary protected calls	optional	routine governance, no migration signal

State precisely what this hides. IP origin and broadcast timing. **Not** on-chain state: the registry entry is public, and it must be, because the guard has to be checkable. You cannot hide *that* you are protected — only *when* and *from where* you acted. Anyone claiming otherwise is selling something.

The mesh is never in the validity path. A compromised relay can delay or drop a transaction; it cannot forge one, because validity is decided by the contract.

3. Threat C — PQ wallets, accounts with no key to expose

The structural point that makes this cheap:

An EOA's address is derived from its public key. A smart account's address is derived from (factory, salt, initcode). **There is no keypair for the account itself** — the validator holds the keys.

So a 4337/7579 account running PQValidator is not "nonce 0 until it signs." It has no secp256k1 public key to expose, ever, and its authorization scheme can be replaced later without the address changing. This is the correct destination for the "everything else" migration tier, and it is why pqguard-spec.md §6.2 calls 7579 the cleanest integration: the signature field is arbitrary bytes by construction.

If ECDSA is kept as one AND factor, that owner key does leak through UserOp calldata on first use. That is fine and expected — AND composition means breaking it alone authorizes nothing.

EOA	4337 account + PQValidator
address = f(pubkey)	address = f(factory, salt, initcode)
key exposed on first send	no account keypair exists
auth scheme fixed forever	validator swappable, address stable
Shor target	no Shor target

4. Threat A — ring-authorized authority

A ring signature on the authority path means an adversary cannot tell which of N keys authorizes the upgrade. They must break **all N**. Cost scales with ring size instead of staying flat.

This is the same primitive Project X uses for payment privacy, pointed at a different problem: **target denial, not payment anonymity**.

4.1 It plugs into the existing socket

`IPQVerifier` is already scheme-agnostic. A ring verifier is a new `schemeId` registration behind the existing governance delay, not a redesign:

- `pkCommitment` becomes a **Merkle root over the ring's member commitments**
- `signature` becomes the ring signature plus its membership argument

Two methods are added to the interface so the registry can tell ring schemes apart without breaking the existing FORS+C verifiers:

```
interface IPQVerifier {
    function verify(bytes32 digest, bytes calldata signature, bytes32 pkCommitment)
        external view returns (bool ok, bytes32 revealedNext); // unchanged

    function schemeId() external pure returns (uint32); // unchanged
    function maxUsesRecommended() external pure returns (uint32); // unchanged
    function isFewTime() external pure returns (bool); // unchanged

    function isRing() external pure returns (bool); // NEW
    function nullifierOf(bytes calldata signature) external pure returns (bytes32); // NEW
}
```

Non-ring verifiers return `false` from `isRing()`, and the registry never calls `nullifierOf` on them.

4.2 The one real state change: nullifiers, not counters

`useCount` is a monotonic per-account counter bound into every digest. **That breaks under a ring**, because you deliberately do not learn which member signed, so there is no counter to increment. Replay protection moves to a nullifier set:

```
struct KeyState {
    bytes32 pkCommitment; // for ring schemes: the ring-set Merkle root
    bytes32 nextCommitment;
    uint32 schemeId;
    uint32 useCount; // single-signer schemes only
    uint32 maxUses;
    uint32 ringEpoch; // NEW: increments when the ring set rotates
    uint64 rotationDeadline;
    uint64 disableAfter;
}

// account => nullifier => consumed
mapping(address => mapping(bytes32 => bool)) public spent;
```

Digest construction for a ring scheme swaps `useCount` for `ringEpoch`:

```
digest = keccak256(abi.encode(
    PQ_DOMAIN, block.chainid, address(core), account,
    schemeId,
    ringEpoch,          // replaces useCount; changes when the set rotates
    keccak256(payload)   // target + calldata, unchanged
))
```

Linkability must be scoped to the digest, not to the key. Monero-style key images are globally linkable — one image per key, forever — which here would mean a member could act exactly once in their lifetime. What is wanted is a tag over `(member key, digest)`, so the same member may authorize a different action later while two attempts at the *same* action collide and the second is rejected.

Getting this backwards produces a guard that bricks itself on the second governance vote, so it is worth stating in the verifier's own tests.

4.3 Ring set rotation

The set rotates the way keys do, and for the same reason: **authenticated by the ring itself, never by ECDSA**. A new root is submitted with a ring signature valid under the *current* root; `ringEpoch` increments, which invalidates every unspent nullifier scoped to the old epoch.

4.4 The tradeoff, stated plainly

A Safe publishes its owners deliberately — that is accountability. A ring hides which member acted, which is **good for target denial and bad for governance transparency**. This is a per-key decision, not a per-protocol one:

Key role	Scheme	Reasoning
Protocol admin, must be attributable	FORS+C, single signer	an auditor has to know who upgraded
Break-glass recovery key	ring	attribution is worthless, target denial is everything
Treasury multisig	ring, if the DAO accepts unattributable spends	genuinely contested; decide before building
Oracle / bridge attestation set	ring	the signer set is the published target list

5. Cost, and why it decides the deployment

Ring verification is the expensive item, and the number is the one already measured for the shielded pool:

Single-signer lattice verification (ETHFALCON/ETHDILITHIUM precedent)	1.9M – 8.8M gas
Ring of 8, if linear in ring size	15M – 70M gas
Ethereum mainnet block gas limit	60M (measured)
FORS+C single-signer, for comparison	~35k gas

Three consequences, in order:

1. **Ring authority belongs on Arc or another L2**, where settlement already happens. Nothing requires it on mainnet.
2. **Ring size is a dial, not a promise**. Ring 4 roughly halves it.
3. **Scope it to keys that act rarely**. A break-glass key used approximately never can afford 15M gas. A treasury multisig voting weekly cannot.

The 89,000-gas figure for ordinary FORS+C protected calls is unchanged. Nothing in this layer makes the common path more expensive.

6. How the three compose

```

npx nonce0 scan 0xProtocol
  | finds the exposed admin key AND the sorted target list
  ▼
nonce0 init --scheme fors-c          single-signer, attributable
nonce0 init --scheme ring --set 8    target denial, break-glass keys
  | registered through the relay mesh: no timing signal
  ▼
nonce0 init --enable                  PQOwnableAdapter becomes the owner
  | the 3-transaction install window, also through the mesh
  ▼
nonce0 migrate --arm                  destination pinned, ECDSA still works
  |
  | ..... Q-day .....
  ▼
nonce0 migrate --execute              PQ-signed; no ECDSA in this path
  | destination: a 4337 account with PQValidator (no keypair to expose)
  | or a shielded note (unlinked)
  ▼
nonce0 ring 0xPool                   did the ring actually buy anonymity?

```

The last step is the honest one. A ring of 8 whose members' keys are already exposed has an effective anonymity of 2, and **that failure applies to authority rings exactly as hard as to payment rings**. The scanner measures it either way; it is an upper bound, capturing anonymity lost to key exposure only.

7. What this does not solve

Unchanged from the existing threat model, and worth repeating because this layer invites overclaim:

- **Retroactive decryption.** Anything encrypted and already on chain is lost.
- **Immutable verifier soundness.** Containment only.
- **A global passive network adversary.** The mesh assumes one does not exist.
- **Compromise of every relay hop.** Network-origin privacy only; never in the validity path.
- **Rings whose members are already exposed.** Target denial fails exactly where the scanner says it fails.

And the one specific to this layer: **on-chain protection status is public.** The registry must be readable for the guard to be checkable. Network privacy hides timing and origin, never the record.

8. Build order, if this is taken up

1. `isRing()` / `nullifierOf()` added to `IPQVerifier`; the nullifier mapping and `ringEpoch` added to `PQKeyRegistry`. Fuzz the ring path against a **mock ring verifier** before any real ring cryptography exists — the state machine is the risk, exactly as it was for the single-signer path.
2. Digest-scoped linkability test: same member, same action, twice → second rejected. Same member, different action → accepted. This is the test that catches the Monero-style mistake.
3. `PQValidator` for 7579, giving the keyless-account destination. Cheapest real win in this document after network privacy.
4. Relay broadcast wired into `nonce0 init / arm / execute`.
5. A real ring verifier, on Arc, sized by measurement rather than by ambition.